

# SPECHUNTER: A Fail-Closed Agent Loop for Security Discovery and Repair on BOOM RTL

Ariel Khais  
University of Washington  
akhais@uw.edu

Ryan Chen  
University of Washington  
ryanc175@uw.edu

Cambell Xiao  
University of Washington  
cambell@uw.edu

**Abstract**—We present SPECHUNTER, a CHIA loop in which LLM agents propose attack programs, repairs, and follow-up attacks against the BOOM out-of-order RISC-V core, while a deterministic validator decides every verdict from cycle-accurate simulation. The design premise is that an agent’s claims are cheap and its mistakes are plausible, so the loop’s value lies in what it refuses to accept. Our flagship result is a practical one: applied to open BOOM bug report #715, SPECHUNTER rebuilt the exact reported revision, replayed the reporter’s own program, and used automated waveform tracing to show that the address taken as evidence of a leak comes from an unrelated load while the dependent load is stopped by an existing register-read gate. The report does not reproduce, and four candidate RTL repairs are all correctly rejected rather than credited. The same loop, driven live by Gemini 2.5 Flash-Lite, found an intentionally seeded cache leak on real BOOM, minimized it, repaired it, and verified attacker exhaustion; a held-out corpus separated vulnerable and repaired variants in all 64 executions. The entire study ran inside a single \$300 GCP free-trial allocation, requested no additional funding, and spent on the order of \$10, with under \$0.05 of model calls exactly accounted by sealed ledgers.

## I. INTRODUCTION

Transient-execution attacks [1], [2] arise from interactions between speculation, privilege checks, and cache state that are hard to enumerate by hand. LLM agents can write plausible attack programs and RTL patches quickly, which makes them attractive for security red-teaming of open cores. They are also very good at producing *plausible but wrong* conclusions: a repair that “fixes” a leak that never occurred, a minimized witness that no longer exercises the threat, or a clean retest that is clean because the attack became irrelevant. In hardware security these mistakes are expensive: a single simulator run of an out-of-order core takes seconds to minutes, and a false “fixed” verdict can send a design team chasing a bug that was never there.

SPECHUNTER is a CHIA [3] loop for the BOOM core [4] in Chipyard [5], built around one rule: *agents propose, the validator disposes*. Agents never emit executable patches or observations; they select from closed, reviewed program operations and repair identifiers, and every verdict is recomputed from raw simulator output bound by SHA-256 to the exact source, binary, and runner that produced it. The loop is deliberately conservative: any result that is not a deterministic, secret-dependent, architecturally silent leak is treated as inconclusive rather than clean, so the loop fails closed. Our contributions are:

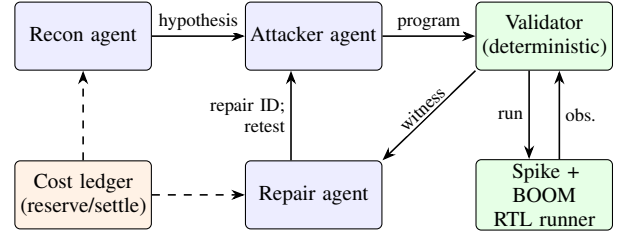


Fig. 1. The SPECHUNTER loop. Blue: LLM agents (Gemini 2.5 Flash-Lite via Vertex AI). Green: trusted, deterministic components that decide every verdict.

- An automated resolution of open upstream BOOM issue #715 [6]: on the exact reported revision the reported leak does not reproduce, and trace analysis localises why (§III).
- A four-agent CHIA loop (recon, attacker, validator, repair) with mandatory witness retest and a strict attacker-exhaustion criterion, running against cycle-accurate BOOM with Spike [7] as the architectural reference and every number bound into a recomputable seal (§II, §IV).
- A catalogue of eleven verifier defects found during live runs, each now covered by a regression test (§V).
- An efficiency result: real-BOOM evidence produced for on the order of \$10 of a \$300 free trial, roughly a tenth of the smallest budget the hackathon’s funding track anticipated (§VI).

## II. LOOP DESIGN

**Agents.** Figure 1 shows the loop. The recon agent states a hypothesis against a security invariant (user code must not observe a PMP-protected secret, architecturally or through cache timing). The attacker turns it into a program over a whitelist of operations (e.g., `enter_user`, `train_branch`, `load_secret`, `encode`, `probe`); the trusted runner compiles these to fixed, reviewed RV64 instruction templates. The repair agent may only choose a closed repair ID; it cannot emit patch text. This closed interface is what lets an untrusted model drive a privileged experiment: the agent chooses *which* reviewed building blocks to combine, never the machine code that runs.

**Threat model and harness.** The invariant is user/supervisor isolation: code in user mode must not learn a byte of a PMP-protected supervisor page, either architecturally or through a

timing side channel. The reviewed harness builds on the RISC-V test/HTIF substrate. It reserves an aligned 4 KiB secret page, configures PMP to deny it, installs a machine-mode trap handler, and enters user mode with `mret`. A denied load and its dependent encode/squash window are skipped architecturally after the load-access fault (cause 5); a fixed probe then times the same two public cache lines in the same order and reports only whether line 0 was faster. Neither the probe addresses nor its control flow depend on the secret, so the probe itself cannot manufacture a signal. Spike and BOOM must agree on architectural output and trap events before a BOOM observation is accepted; any disagreement, unexpected trap, or timeout is inconclusive.

**Validator.** Each candidate runs in four executions: two secret values, each repeated twice. A violation requires a deterministic, secret-dependent observation with the expected load-access fault and no architectural secret value; anything else is *inconclusive*, never clean. Findings are minimized by delta debugging, and the minimized program must itself reproduce before it is recorded.

**Repair acceptance.** After a repair, the orchestrator (not the model) replays the exact minimized witness, which must be clean. Control then returns to the attacker. A repair is verified only on attacker exhaustion, and in the current schema exhaustion additionally requires a distinct, threat-relevant challenge that violates on the *unrepaired* variant and is clean on the repaired one. Any later violation revokes verification.

**Provenance.** Chipyard 1.14, BOOM, and toolchains are pinned by commit and hash; the runner refuses dirty trees or mismatched binaries. Every evidence file is bound into a SHA-256 seal, and a single verifier recomputes all 43 file bindings across the ten seals and rebuilds the issue #715 case seal from its raw waveforms; continuous integration runs it on every change. SPECHUNTER runs as a decorated CHIA node on a local Ray runtime.

### III. RESOLVING UPSTREAM BOOM ISSUE #715

Issue #715 [6] reports that on a historical BOOM a supervisor-page load that faults can still forward data to a dependent load during speculation, leaking it through the TLB. It is exactly the kind of ambiguous, high-stakes report where an automated, evidence-bound loop is useful. We tested it three ways.

**Adaptation.** A reviewed adaptation of the mechanism was deterministic and clean on the current pin (4 executions) and on the exact reported Chipyard 004297b6/BOOM fac2c370 with historical SmallBoomConfig (built in 685 s).

**Original attachment.** We fetched the reporter’s ELF by pinned URL and digest, converted its entry-zero image to a fixed load image, and captured a VCD waveform at a fixed seed. Table I summarizes the gadget. A scanner joins events by ROB index, physical register, load-queue slot, and branch mask, and requires one ordered chain within the first speculation window. The trace shows the trained branch (cycle 3769), a branch-masked protected-page TLB request (3807),

TABLE I  
ISSUE #715 ATTACHMENT GADGET (WRONG PATH) AND WHAT THE HISTORICAL-RTL WAVEFORM SHOWS FOR EACH LOAD.

| Offset | Instruction      | Observed in waveform                  |
|--------|------------------|---------------------------------------|
| +0     | lb sp, -2048(tl) | protected-page TLB req. (3807); fault |
| +4     | ld s1, 0(sp)     | issues poisoned; register read gated  |
| +8     | lb s1, 1439(a0)  | requests 0x59f (from 3809)            |

requests to 0x59f (from 3809), a page fault, and misprediction resolution (3910). At first sight this looks like the reported leak. However, the disassembly shows that 0x59f is the address of an independent third load (`lb s1, 1439(a0)` with a zero base;  $1439 = 0x59f$ ), not a secret-derived address. The dependent load at gadget offset +4 issues with its source operand poisoned while the LSU reports a load miss, and BOOM’s register-read gate (`exu/core.scala`, lines 973–978) drops its issue-valid, so it never presents a valid LSU request.

**Repairs correctly rejected.** Before this attribution was clear, we iterated on four hash-pinned candidate LSU patches outside the closed repair list, each driven by the previous matched-trace comparison: v1 same-cycle fault gates; v2 blocking loads under unresolved branches; v3 suppressing fast wakeup and excepting-entry writeback; v4 requiring a registered D-cache fire before speculative wakeup. V1–v3 left the 0x59f chain cycle-identical to baseline, and v4’s waveform was not recovered. Because the baseline never demonstrated the dependent-load mechanism, *no* repair can be credited, and every comparison seal records `security_fix_validated: false`. A loop that accepted “the patch built and nothing leaked” would have declared success four times here. The practical takeaway for the BOOM project is that this report, on the revision as filed, does not exhibit the described leak, and the exact mechanism that suppresses the dependent load is identified.

### IV. EVALUATION

All BOOM runs used GCP e2-standard-8 workers with no service account, a six-hour deletion cap, and explicit deletion after evidence recovery. Table II lists every result together with the kind of target it was measured on; we do not combine results across kinds.

**Real BOOM baseline and live run.** A reviewed payload configures PMP to deny a 4 KiB page, drops to user mode via `mret`, and requires a load-access fault (cause 5); the same ELF passed on Spike and SmallBoomV3, and two programs (architectural denial and a transient window) were clean and deterministic on the pristine core. Because pristine BOOM exhibited no violation, we added a clearly labelled positive control that touches a secret-selected cache line before entering user mode; it produced probe sequences `[1], [0], [1], [0]` before repair and all zeros after, over 16 executions on the *same* simulator binary. Driven live on this control, Gemini 2.5 Flash-Lite proposed a six-operation program, BOOM confirmed the leak, minimization kept

TABLE II

SEALED EVIDENCE BY TARGET TYPE. “MUTATION” IS AN INTENTIONALLY SEEDED HARNESS LEAK, NOT AN UPSTREAM BOOM BUG.

| Experiment                        | Target        | Result                                      |
|-----------------------------------|---------------|---------------------------------------------|
| PMP privilege gate (Spike + BOOM) | BOOM RTL      | pass, trap 5, no leak                       |
| Pristine baseline matrix          | BOOM RTL      | 8/8 clean                                   |
| Seeded positive control           | BOOM+mutation | 8/8 violating; 8/8 clean after repair       |
| Live Vertex loop                  | BOOM+mutation | found, repaired, exhausted; \$0.0006        |
| Held-out corpus (8 programs)      | BOOM+mutation | 64/64 correct                               |
| LSU fault-gate RTL patch          | patched BOOM  | builds; 8/8 clean; <i>no security claim</i> |
| Issue #715, current + historical  | BOOM RTL      | not reproduced                              |
| Guided vs. random search          | Fixture       | see Table III                               |
| 10× loop repeatability            | Model fixture | 10/10; \$0.0054 total                       |
| Loop through CHIA node            | Model fixture | complete; \$0.0005                          |

TABLE III

SEARCH ON DETERMINISTIC FIXTURES (PRIVILEGE, TRANSIENT-CACHE, AND SECURE-CONTROL BENCHMARKS; 16-ITERATION BUDGET).

|                                  | Guided | Random (1000 seeds) |
|----------------------------------|--------|---------------------|
| Positive cases found             | 2/2    | 1145/2000 (57.3%)   |
| 95% Wilson interval              | —      | 55.1–59.4%          |
| Mean attempts to find            | 1.5    | 8.48                |
| False positives (secure control) | 0      | 0/1000              |

`enter_user`, `load_secret`, `probe`, the repair agent chose the closed repair, the witness retested clean, and the attacker reached exhaustion: 28 BOOM executions, 4 Vertex calls, zero inconclusive cases, and \$0.0006038 of accounted cost.

**Held-out corpus and candidate patch.** Eight fixed programs vary training, encoding, squash and fence placement, and noise, all retaining the protected load. With two repetitions in both secret worlds, all 32 mutated executions were deterministic violations and all 32 repaired executions were clean — a test that the *validator and repair gate* generalize across program shape, not an RTL fix. Separately, source review of BOOM’s `lsu.scala` found that incoming and retried D-cache requests are not gated by same-cycle `ae_ld/pf_ld/ma_ld`; a two-path gate patch built into a distinct simulator and ran the secure matrix cleanly (8/8). Since the baseline was already clean, this shows only that the patch builds and does not break the target tests; the seal records `security_fix_validated: false`.

**Search and repeatability.** Table III compares the deterministic guided policy with random programs over three small fixtures; the guided arm is a single deterministic run, a sanity check on search structure, not a statistical claim about LLM agents. Separately, ten independent Gemini-driven trials on a fast model of the positive control all completed discovery, repair, retest, and exhaustion (40 calls, 264 fixture executions, \$0.0053752), and a run through the CHIA `run_agent_experiment` node (CHIA 1.0.1, Ray 2.54.0) completed the same lifecycle for \$0.0005130.

TABLE IV

VERIFIER DEFECTS FOUND AND CLOSED. FP = WOULD HAVE ENABLED A FALSE POSITIVE (A FALSE FINDING OR FALSE “FIXED”).

| Defect                                                         | Effect       |
|----------------------------------------------------------------|--------------|
| Minimizer could drop the protected load from a witness         | FP finding   |
| Retest of the same program counted toward exhaustion           | FP fix       |
| Irrelevant (e.g. probe-only) challenge counted as clean        | FP fix       |
| Challenge clean on <i>both</i> variants counted as clean       | FP fix       |
| Verified flag survived a later-cycle violation                 | FP fix       |
| Stale clean-replay flag carried across recon cycles            | FP fix       |
| Short observation batch skipped repeatability check            | FP either    |
| VCD scanner read intermediate combinational values             | FP finding   |
| Unrelated trace events could form a “mechanism chain”          | FP finding   |
| Seed equality inferred from matching timing                    | provenance   |
| Git porcelain <code>.strip()</code> rejected valid repair tree | false reject |

## V. WHAT THE LIVE RUNS TAUGHT US

Table IV lists verifier defects exposed by live runs or review of their transcripts. Each would have produced an unsupported claim. Each is now covered by a regression test.

Three lessons generalize. First, *a clean result is only evidence if the same test fails on the vulnerable variant*; four of the defects are versions of this. Second, minimizers and scanners are adversarial surfaces: they optimize for the property check, not the threat model, and must re-encode the threat model as constraints. Third, a negative result on real RTL (issue #715) required more infrastructure than the positive control, and it is also the more informative of the two.

## VI. COST AND EFFICIENCY

SPECHUNTER was built and evaluated inside a single \$300 GCP free-trial allocation and requested no additional funding at any point. All model usage is exactly accounted by the sealed cost ledgers and totals well under \$0.05 across every sealed and development call: the live BOOM discovery-and-repair loop cost \$0.0006, the ten repeatability trials \$0.0054, the CHIA-node run \$0.0005, and the early integration smoke tests \$0.0047. Cloud compute was a small number of short-lived, auto-deleting workers, each rebuilt from pinned sources and deleted after its evidence was hashed; billing is delayed, but their recorded runtimes put total project spend on the order of \$10. For context, the hackathon’s funding proposals were asked to target \$100–\$1,500; SPECHUNTER produced real-BOOM RTL evidence, a live agentic loop, and the issue #715 resolution for roughly a tenth of the low end. The dominant cost is CPU time for cycle-accurate Verilator builds, not model inference, which is a useful data point for anyone budgeting agentic RTL security work: the LLM is nearly free relative to the simulator.

## VII. DESIGN BOUNDARIES, LIMITATIONS, AND FUTURE WORK

We found no new BOOM vulnerability; every successful discovery and repair is on an intentional harness mutation, the live BOOM loop ran once, and the search comparison uses three small fixtures. Two of these limits are deliberate design boundaries rather than accidents. First, the attacker is

confined to a closed set of reviewed operations that the trusted runner compiles into fixed templates, and the repair agent selects only closed identifiers. This boundary is simultaneously a soundness mechanism — no unreviewed code decides a verdict — and a safety mechanism, since an untrusted model never executes arbitrary privileged code on the core; it also bounds the search space to combinations of primitives the loop has been given. Second, general-purpose LLM assistants apply constraints to offensive-security tasks, which, together with a hackathon-scale time and compute budget, bounded how far we pushed the autonomous attacker within the event. The architecture, however, is unchanged by scale: widening the reviewed primitive set (more speculation shapes, more covert-channel constructions, a richer observation channel) and adding compute is the direct path to exercising broader transient-execution behaviour, and reproducing a *published* vulnerability class is the natural next milestone before anything novel. Crucially, that path does not weaken the guarantee that matters: the differential oracle and the recomputable seals, not the model, would still decide every verdict.

### VIII. RELATED WORK

SPECHUNTER sits at the intersection of three lines of work. *Agentic discovery loops* such as AlphaEvolve [8] and VibeServe [9] show that an LLM in an outer search loop with an inner correctness or performance check can improve algorithms and build bespoke systems; SPECHUNTER applies the same propose-then-check structure to microarchitectural security and runs inside CHIA [3], which expresses such flows as directed cyclic graphs over hardware tools. *Red-team LLM agents* such as PentestGPT [10] and the agents evaluated by Cybench [11] autonomously find and exploit software vulnerabilities; SPECHUNTER is a blue-team counterpart at the RTL level, where that same autonomy is dangerous unless a deterministic oracle gates every claim. *Hardware fuzzing* drives RTL toward coverage without a security oracle (RFUZZ [12]); differential fuzzing adds one by comparing a design against itself under different secrets, the strategy our validator follows, and hardware–software contracts [13] formalise what speculation may expose. SPECHUNTER’s specific contribution is the fail-closed evidence discipline: an untrusted model may steer the search, but no claim survives that a deterministic oracle and a recomputable seal do not independently support.

### IX. CONCLUSION

SPECHUNTER shows that an LLM-driven security loop on real out-of-order RTL is cheap to run — on the order of \$10 end-to-end, with the model nearly free — and that its credibility comes from a validator that fails closed. Its most practical result is the automated resolution of open BOOM report #715. The loop, pinned BOOM runners, and every sealed artifact are open source at <https://github.com/Ryclic/SpecHunter>.

### AI ASSISTANCE DISCLOSURE

Gemini 2.5 Flash-Lite is a component of the evaluated loop. AI coding assistants (OpenAI Codex, Google Gemini/Antigravity, and Anthropic Claude Code) were used to

write and review code and tests, analyze evidence, and draft and edit this paper. The human authors reviewed all claims against the sealed evidence and are responsible for the paper’s content.

### REFERENCES

- [1] P. Kocher *et al.*, “Spectre attacks: Exploiting speculative execution,” in *IEEE Symp. Security and Privacy (S&P)*, 2019.
- [2] M. Lipp *et al.*, “Meltdown: Reading kernel memory from user space,” in *USENIX Security Symp.*, 2018.
- [3] A. Cui *et al.*, “CHIA: An open-source framework for principled, agentic AI-driven hardware/software co-design research,” arXiv:2606.27350, 2026.
- [4] C. Celio, P.-F. Chiu, B. Nikolić, D. A. Patterson, and K. Asanović, “BOOM v2: An open-source out-of-order RISC-V core,” Univ. California, Berkeley, Tech. Rep. UCB/EECS-2017-157, 2017.
- [5] A. Amid *et al.*, “Chipyard: Integrated design, simulation, and implementation framework for custom SoCs,” *IEEE Micro*, vol. 40, no. 4, pp. 10–21, 2020.
- [6] T. Kovats, “[BUG REPORT] Speculative meltdown-type leakage through the TLB, riscv-boom issue #715,” 2025. [Online]. Available: <https://github.com/riscv-boom/riscv-boom/issues/715>
- [7] RISC-V International, “Spike RISC-V ISA simulator.” [Online]. Available: <https://github.com/riscv-software-src/riscv-isa-sim>
- [8] A. Novikov *et al.*, “AlphaEvolve: A coding agent for scientific and algorithmic discovery,” arXiv:2506.13131, 2025.
- [9] K. Kamahori, S. Li, S. Peter, and B. Kasikci, “VibeServe: Can AI agents build bespoke LLM serving systems?,” arXiv:2605.06068, 2026.
- [10] G. Deng *et al.*, “PentestGPT: An LLM-empowered automatic penetration testing tool,” in *USENIX Security Symp.*, 2024.
- [11] A. K. Zhang, N. Perry *et al.*, “Cybench: A framework for evaluating cybersecurity capabilities and risks of language models,” in *Int. Conf. Learning Representations (ICLR)*, 2025.
- [12] K. Laeuffer, J. Koenig, D. Kim, J. Bachrach, and K. Sen, “RFUZZ: Coverage-directed fuzz testing of RTL on FPGAs,” in *IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2018.
- [13] M. Guarnieri, B. Köpf, J. Reineke, and P. Vila, “Hardware-software contracts for secure speculation,” in *IEEE Symp. Security and Privacy (S&P)*, 2021.